



DATA STRUCTURES

CONTENTS



39. Shortest Path Algorithm – I

TV. GEETHA



Welcome to the e-PG Pathshala Lecture Series on Data Structures. In this module we will discuss about Shortest Path Algorithm.

Learning Objectives

The learning objectives of the module are as follows:

- To understand the Shortest Path problem and its variants
- To discuss the properties of Shortest path
- To explain the Dijkstra's Algorithm
- To illustrate the Dijkstra's Algorithm with an example

39.1 Recap

Topological sort is a method of arranging the vertices in a directed acyclic graph (DAG), as a sequence, such that no vertex appear in the sequence before its

predecessor. The properties of the Topological sort and the step -by-step description had been discussed with an example.

39.2 Shortest Path Problem

In this module we discuss the shortest path algorithm. Shortest Path Problem is usually associated with a weighted graph that has two vertices u and v , and we want to find a path of minimum total weight between u and v . Length of a path is the sum of the weights of its edges.

Example: Shortest path between Bangalore and Madurai. An example of a graph for shortest path is given in Figure 39.1. Applications of shortest path problem include Internet packet routing, Flight reservations and Driving directions.

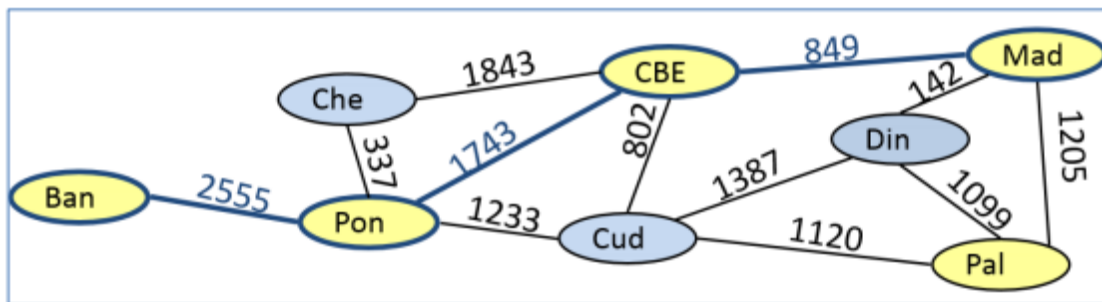


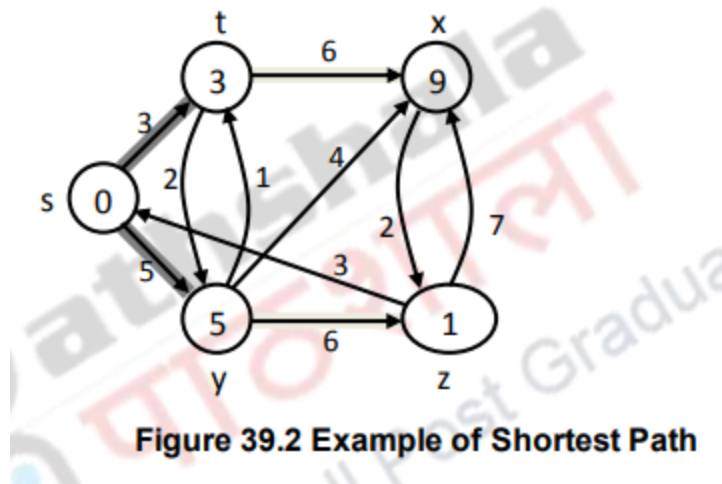
Figure 39.1 Example of Shortest Path

The shortest path algorithm is associated with a directed weighted graph and the path length is sum of weights of the edges on the path. The source vertex is the place where the path begins and the destination vertex is the vertex where the path ends. Once the source and destination are finalized the vertices in the increasing order is reported from the source vertex. We construct the shortest path edge by edge; at each step adding one new edge, corresponding to construction of shortest path to the current new vertex.

39.2.1 Shortest Path Notations

For each vertex $v \in V$ where V is the set of vertices. $\delta(s, v)$ is shortest-path weight

and $d[v]$ is the shortest-path weight estimate. Initially, $d[v] = \infty$ and $d[v] \neq \delta(s, v)$ as algorithm progresses. $p[v] = \mathbf{predecessor}$ of v on a shortest path from s and if no predecessor exists, $p[v] = \text{NIL}$. p induces a tree called **the shortest-path tree as shown in Figure 39.2.**



39.3 Shortest Path Algorithm

The input to the algorithm is the Directed graph $G = (V, E)$, with V being the set of vertices and E being the edges. A weight function $w : E \rightarrow \mathbb{R}$ is associated with each edge of the graph. A path is denoted by a sequence of vertices. The weight of path $p = \langle v_0, v_1, \dots, v_k \rangle$, is the weight of the edges between the vertices in the path. The Shortest-path weight from u to v : $d(u, v) = \min w(p) : \text{if there exists a path } p \text{ from } u \text{ to } v \neq \infty : \text{otherwise}$. Note that there might be multiple shortest paths from u to v . There may be many types of shortest path algorithms.

39.3.1 Initialization

All the shortest-paths algorithms start with INITIALIZE-SINGLE-SOURCE. Initially for every vertex v in G , $d[v]$ the shortest path estimate is set to ∞ and the $p[v]$, the predecessor of every vertex v is set to Nil. $d[s]$, the shortest path estimate of source node is set to 0.

Alg.: INITIALIZE-SINGLE-SOURCE (V, s)

for each $v \in V$

do $d[v] \leftarrow \infty$

$\pi[v] \leftarrow \text{NIL}$

$d[s] \leftarrow 0$

39.2.2 Shortest Path Tree

For every node $v \in V$, $\pi[v]$ is the predecessor of v in shortest path from source s to v . This value is set to Nil if does not exist. All our algorithms will output a shortest-path tree whose root is the source s and the edges are $(\pi[v], v)$. The shortest path between s and v is the unique tree path from root s to v . Consider Source node 1 and destination node 7, a direct path between the nodes will cost 14, however a shorter path through two other nodes costs only 11 (Figure 39.3).

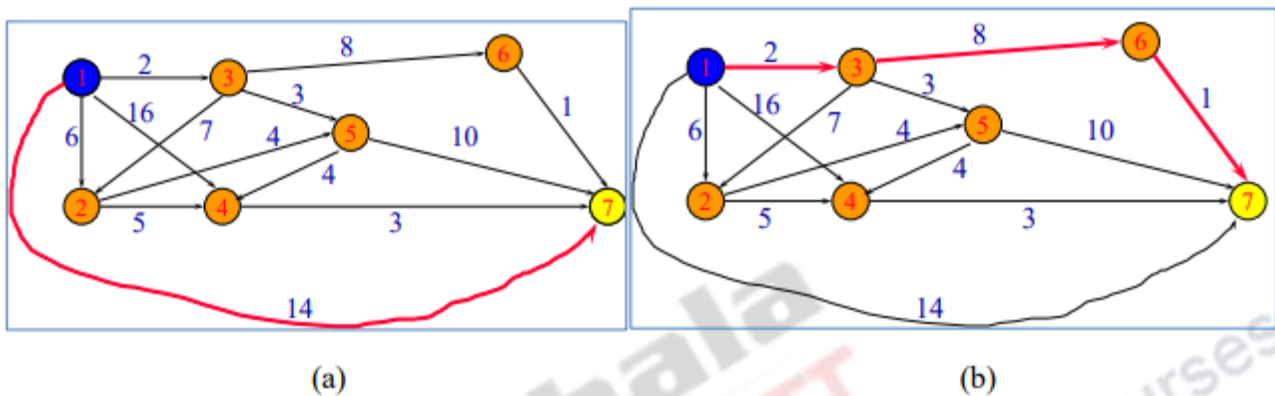


Figure 39.3 Shortest Path

39.3 Shortest Path Variants

There are many variants of the shortest path algorithms. They are:

Single-source single-destination (1-1): Find the shortest path from source s to destination v .

Single-source all-destination(1-Many): Find the shortest path from s to each vertex v .

Single-destination shortest-paths (Many-1): Find a shortest path to a given destination vertex t from each vertex v .

All-pairs shortest-paths problem (Many-Many): Find a shortest path from u to v for every pair of vertices u and v .

Single source/All destinations -nonnegative edge cost

Need to solve: determine a shortest path from v to each of the remaining vertices of G .

39.3.1 Single-destination shortest paths

Among the shortest path algorithms given above the single destination shortest paths is about finding a shortest path to a given destination vertex t from each vertex v . Here we reverse the direction of each edge and finding the distances between every pair of vertices in a weighted directed graph G . Figure 39.4 shows the example of the **Single source/All destinations** -nonnegative edge cost where we need to determine a shortest path from v to each of the remaining vertices of G .

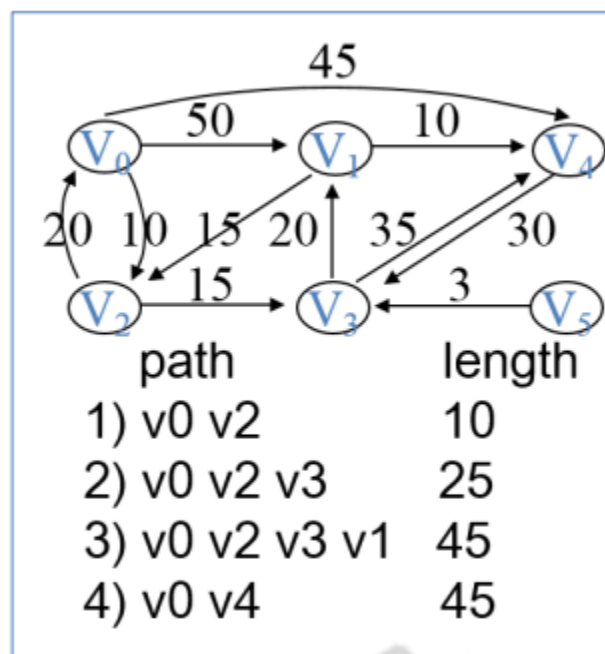


Figure 39.4 Single source/All destinations

39.3.2 Shortest Path Properties

If some path from s to v contains a negative cost cycle, there does not exist a shortest path. Otherwise, there exists a shortest s - v that is simple. A negative cycle can produce arbitrarily long negative paths by traversing cycle a number of times. The negative-weight edges may form negative-weight cycle. If such cycles are reachable from the source, then $\delta(s, v)$ the shortest-path weight is not properly defined .

39.4 Relaxation Process

For each vertex v , we maintain an upper bound $d[v]$ on the weight of shortest path from s to v and $d[v]$ initialized to infinity. **Relaxing** an edge (u, v) = testing whether we can improve the shortest path to v found so far by going through u . $d[v]$ is the weight of the path to vertex v , $d[u]$ is the weight of the path to vertex u , $w[u,v]$ is weight of edge $[u,v]$ and $p[v]$ indicates the predecessor of v in the shortest path to it (Figure 39.5).

If $d[v] > d[u] + w(u, v)$ and we can improve the shortest path to v . The updated $d[v] \Rightarrow d[v] = d[u] + w(u,v)$ and $p[v]$ is updated as $\Rightarrow p[v] \leftarrow u$

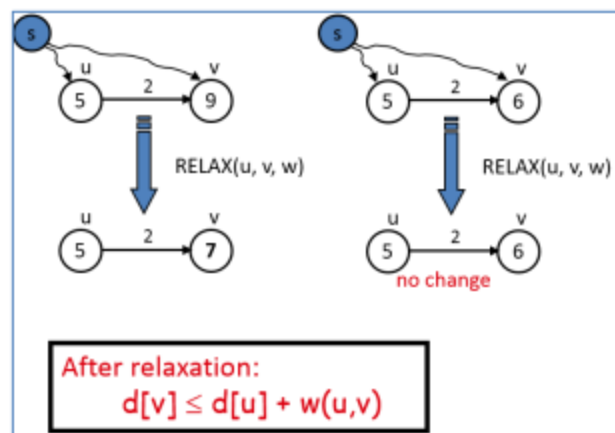


Figure 39.5 Relaxation Step

Relaxing an edge (u, v) consists of testing whether we can improve the shortest path to v found so far by going through u , then updating $d[v]$ and $p[v]$ accordingly. A

relaxation step **may** either **decrease** the value of the shortest-path estimate $d[v]$ and update v 's predecessor $p[v]$, or cause **no change**.

39.5 Shortest Path Properties

Now let us define some properties of shortest paths.

39.5.1 Triangle Inequality

For any edge (u, v) , we have $\delta(s, v) \leq \delta(s, u) + w(u, v)$. The weight of the shortest path from s to v is no greater than the weight of the shortest path from s to u plus the weight of the edge from u to v . For all $(u, v) \in E$, we have:

$$\delta(s, v) \leq \delta(s, u) + w(u, v)$$

If u is on the shortest path to v we have the equality sign (Figure 39.6).

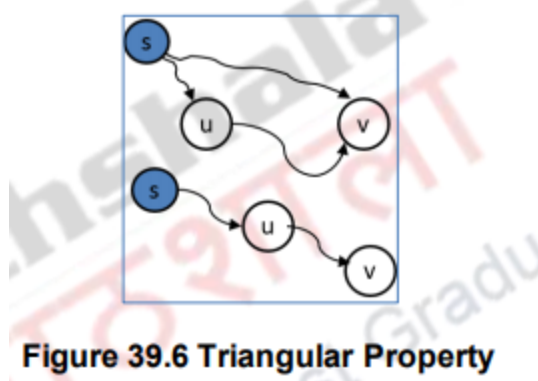


Figure 39.6 Triangular Property

39.5.2 Optimal substructure property

All sub-paths of shortest paths are also shortest paths. The shortest path problem has the following optimal substructure property. If a node x lies in the shortest path from a source node u to destination node v then the shortest path from u to v is combination of shortest path from u to x and shortest path from x to v .

39.5.3 Upper-bound property

We always have $d[v] \geq \delta(s, v)$ for all v . The estimate never rises since the relaxation process is so defined that it only lowers the estimate (Figure 39.7).

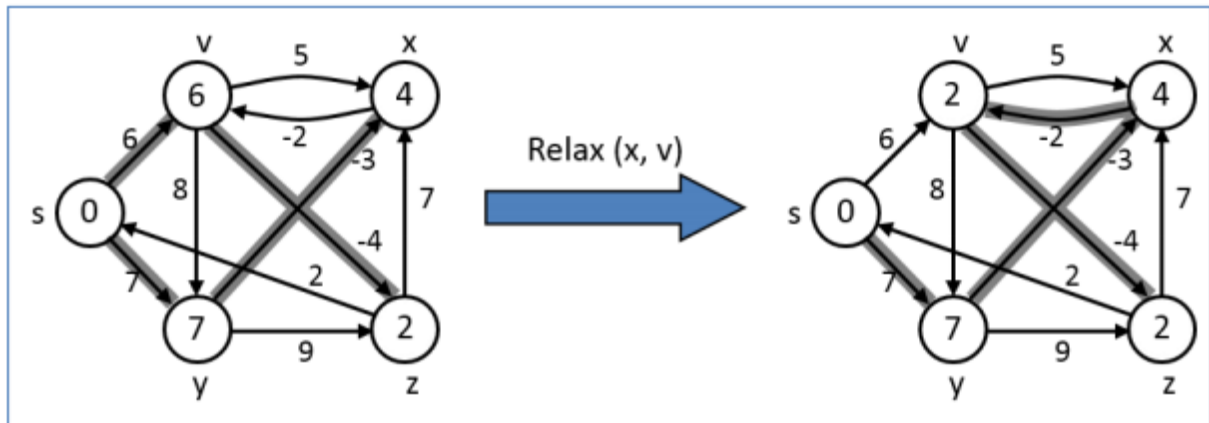


Figure 39.7 Upper-bound Property and Relaxation step

39.5.4 Convergence property

If $s \rightarrow u \rightarrow v$ is a shortest path, and if $d[u] = \delta(s, u)$ at any time prior to relaxing edge (u, v) , $d[v] = \delta(s, v)$ at all times after relaxing (u, v) .

As shown in Figure 39.8, If $d[v] > \delta(s, v)$ after relaxation:

$$d[v] = d[u] + w(u, v) \text{ and } d[v] = 5 + 2 = 7.$$

Otherwise, the value remains unchanged, because it must have already been the shortest path value.

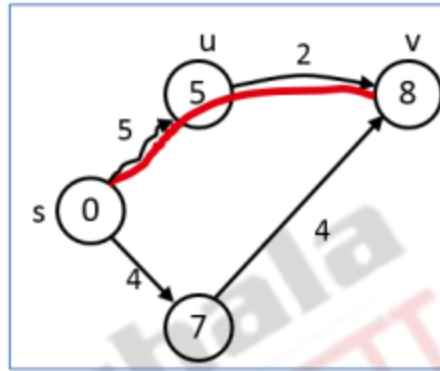


Figure 39.8 Convergence Property

39.5.5 Path relaxation property

Let $p = \langle v_0, v_1, \dots, v_k \rangle$ be a shortest path from $s = v_0$ to v_k . If we relax, in order, $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$, even intermixed with other relaxations, then $d[v_k] = \delta(s, v_k)$.

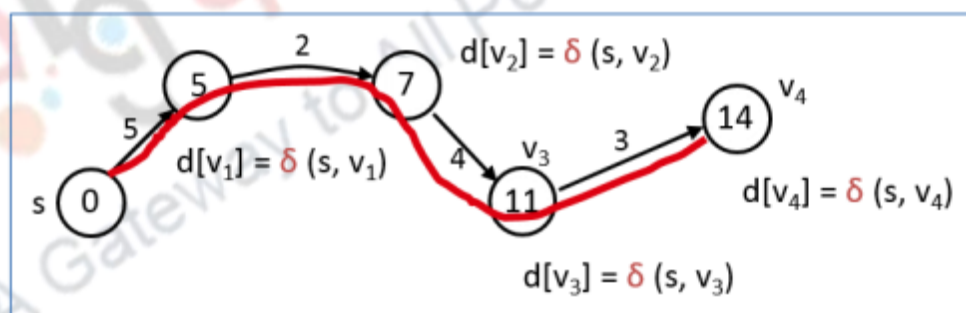


Figure 39.10 Path relaxation Property

39.5.6 Predecessor-subgraph property

Once $d[v] = \delta(s, v)$ for all $v \in V$, the predecessor sub-graph is a shortest-paths tree rooted at s . The Predecessor subgraph $G_\pi = (V_\pi, E_\pi)$, is defined where

$$V_\pi = \{v \in V : \pi[v] \neq \text{Nil}\} \cup \{s\}$$

$$E_\pi = \{(\pi[v], v) \in E : v \in V_\pi - \{s\}\}$$

The No-path property of course says that if there is no path from s to v , then we always have $d[v] = \delta(s, v) = \infty$.

39.6 Applications

Some of the general types of applications of shortest path algorithms include minimization of the total *distance* traveled, minimization of the total cost of a sequence of activities, minimization of the total *time* of a sequence of activities. Application areas include network optimization, packet routing, image segmentation, computer-assisted surgery, computer games, DNA analysis, injection molding, operator scheduling, production planning, re-allocation of resources, approximation of piecewise linear functions and VLSI physical design. As discussed the shortest path algorithms find applications in many diverse areas.

39.7 Relaxation Summary

All algorithms in this module call INITIALIZE-SINGLE-SOURCE and then repeatedly relax edges. Relaxation is the only means by which shortest-path estimates and predecessors change. The shortest path algorithms differ in **how many times** they relax each edge and **the order in which they relax edges**. Bellman-Ford algorithm relaxes each edge many times, while Dijkstra's algorithm for directed acyclic graphs relax each edge exactly once.

39.8 Algorithms

Dijkstra's algorithm and Bellman-Ford algorithm both use the common operations of Initialization and Relaxation. In Dijkstra's algorithm negative weights are not allowed. In the Bellman-Ford algorithm negative weights are allowed but negative cycles reachable from the source are not allowed.

39.9 Dijkstra's Algorithm

The steps and assumptions of the Dijkstra's algorithm are as follows:

1. The distance of a vertex v from a vertex s is the length of a shortest path between s and v .
2. Dijkstra's algorithm computes the distances of all the vertices from a given start vertex s .
3. Assumptions are made that
 - the graph is connected
 - the edges are undirected.
 - The edge weights are nonnegative.
4. Grow a “**cloud**” of vertices, beginning with s eventually covering all the vertices.
5. We store with each vertex v a label $d(v)$ representing the distance of v from s in the subgraph consisting of the cloud and its adjacent vertices.
6. At each step,
 - we add to the cloud the vertex u outside the cloud with the smallest distance label, $d(u)$ and
 - we update the labels of the vertices adjacent to u .

39.9.1 Edge Relaxation

Now let us discuss the edge relaxation procedure in detail. Consider an edge $e = (u, z)$ such that, u is the vertex most recently added to the cloud and z is not in the cloud. The relaxation of edge e updates distance $d(z)$ as follows: $d(z) = \min\{d(z), d(u) + \text{weight}(e)\}$. In other words we consider the edge from u to z that is the minimum (Figure 39.11).

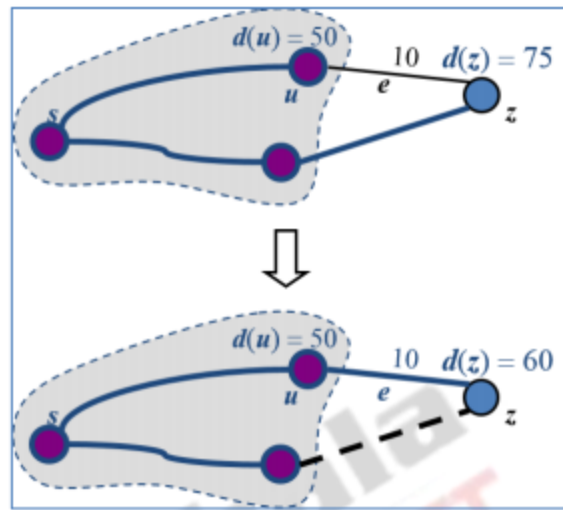


Figure 39.11 Edge Relaxation

Let us consider the example given in Figure 39.12 (i) and (ii) adopted from webhome.csc.uvic.ca/~ruskey/classes/326/slides/Chpt7ShortestPath.ppt

We start with node A and $d[A]$ at this point is 0 Figure 39.12 (i)-a. Now we add to the cloud the vertex u outside the cloud with the smallest distance label, $d(u)$ which here is vertex C and $d[C] = 2$ (A-C). We then update the labels of the vertices adjacent to u , here note that label of vertex D is updated from 4 to 3 (A-C-D), and label of vertex F (A-C-F) is updated to 11. Note that the path to B from A through C (C-B) and path to D from A (A-D) are not the shortest and are marked (in dotted lines) shown in Figure 39.12 (i)-b.

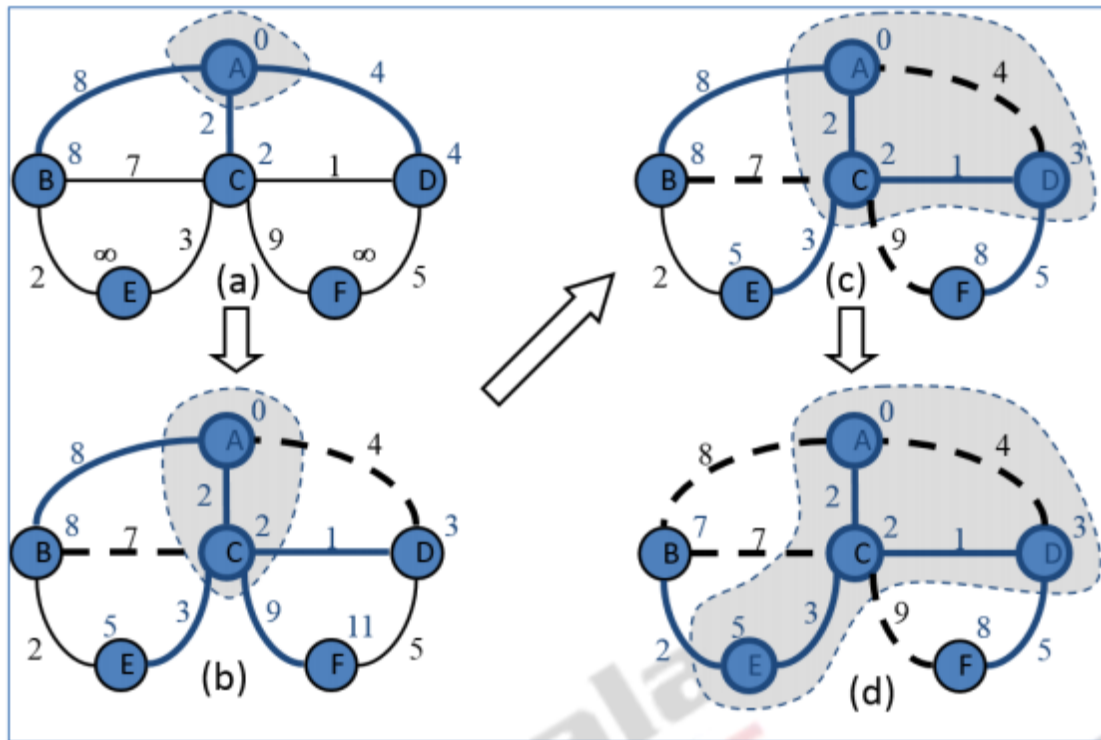


Figure 39.12 (i) Example

Now we add to the cloud the vertex **D** outside the cloud with the smallest distance label, $d[D] = 3$ (A-C-D). We then update the labels of the vertices adjacent to **u**, here note that label of vertex **F** is updated from 11 to 8 (A-C-D-F), and the path that is not the shortest (C-F) is marked (in dotted lines) shown in Figure 39.12 (i)-c.

Now we add to the cloud the vertex **E** outside the cloud with the smallest distance label, $d[E] = 5$ (A-C-E). We then update the labels of the vertices adjacent to **u**, here note that label of vertex **B** is updated from 8 to 7 (A-C-E-B), and the path that is not the shortest (C-F) is marked (in dotted lines) shown in Figure 39.12 (i)-d.

Now we add to the cloud the vertex **B** outside the cloud with the smallest distance label, $d[B] = 7$ (A-C-E-B). We then update the labels of the vertices adjacent to **u**, here no updation occurs (Figure 39.12 (i)-e).

Finally we add the node **F** to the cloud. Again no updation occurs (Figure 39.12 (i)-f).

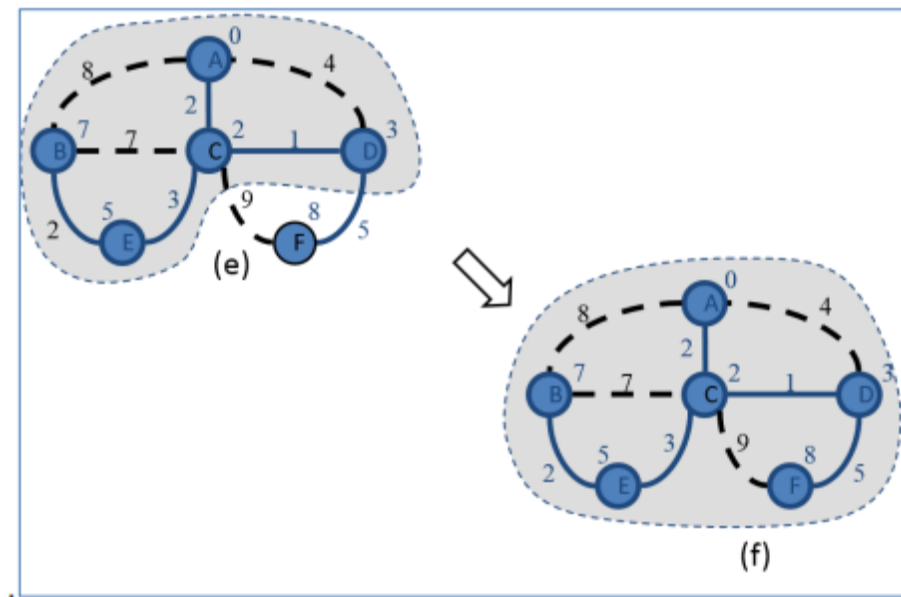


Figure 39.12 (ii) Example

39.9.2 Dijkstra's Algorithm

A priority queue stores the vertices outside the cloud where Key is distance for the element which is the vertex. Locator-based methods are used to insert – **insert(k,e)** which returns a locator and **replaceKey(l,k)** which changes the key of an item. We store two labels with each vertex, Distance ($d(v)$ label) and locator in the priority queue.

Dijkstra's algorithm assumes that $w(e) \geq 0$ for each e in the graph.

We maintain a set S of vertices such that, every vertex $v \in S$, $d[v] = d(s, v)$, i.e., the shortest-path from s to v has been found. (Initial values: $S = \text{empty}$, $d[s] = 0$ and $d[v] = \infty$)

(a) select the vertex $u \in V - S$ such that
 $d[u] = \min \{d[x] | x \in V - S\}$. Set $S = S \cup \{u\}$

(b) **for** each node v adjacent to u **do** RELAX(u, v, w).
 Repeat step the above steps until $S = V$.

The pseudo code for the algorithm is shown below:

```

dist[s] ← 0                (distance to source vertex is zero)
for all v ∈ V - {s}
do dist[v] ← ∞              (set all other distances to infinity)
S ← ∅                      (S, the set of visited vertices is initially empty)  \ Q ← V
(Q, the queue initially contains all vertices)
while Q ≠ ∅                (while the queue is not empty)
do u ← mindistance(Q, dist) (select the element of Q with the min. distance)
S ← S ∪ {u}                (add u to list of visited vertices)
for all v ∈ neighbors[u]
do if dist[v] > dist[u] + w(u, v) (if new shortest path found)
then d[v] ← d[u] + w(u, v) (set new value of shortest path)
return dist

```

39.9.3 Features of Dijkstra's Algorithm

The Dijkstra's algorithm is a *greedy* algorithm. It “Visits” every vertex only once, when it becomes the vertex with minimal distance amongst those still in the priority queue. However distances may be revised **multiple times** since the current values represent only the ‘best guess’ based on our observations so far. Once a vertex is visited we are guaranteed to have found the shortest path to that vertex

39.9.4 Analysis of the Algorithm

The Graph operation is the method `incidentEdges` which is called once for each vertex. The Label operations are the ones where we set/get the distance and locator labels of vertex z that is of the order $O(\deg(z))$ times and setting/getting a label takes $O(1)$ time. The Priority queue operations include insertion where each vertex is inserted once into the priority queue and removed once from the priority queue, where each insertion or removal takes $O(\log n)$ time. The key of a vertex in the priority queue is modified at most $\deg(w)$ times, where each key change takes $O(\log n)$ time.

Thus the Dijkstra's algorithm runs in $O((n + m) \log n)$ time provided the graph is represented by the adjacency list structure – Recall that $\sum_v \deg(v) = 2m$. The running time can also be expressed as $O(m \log n)$ since the graph is connected.

39.10 Dijkstra's Algorithm – Walkthrough

Let us use the graph in Figure 39.13 (a) to find the shortest path using Dijkstra's algorithm. .

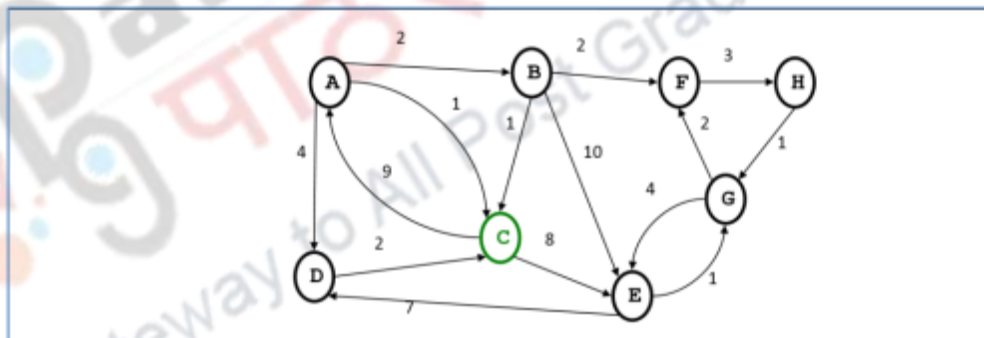


Figure 39.13 (a)

Example has been adopted from

<https://courses.cs.washington.edu/courses/cse326/03wi/.../lecture21b.ppt>.

Initially the distance to all vertices except the source vertex is initialized to ∞ . Let us assume that C is the source vertex $d[C] = 0$ and C is added to the cloud . Now we update labels of neighbors of C – here A is updated to 9 and E to 8. Since distance to E is the shortest from C that will be the next node to be considered (Figure 39.13 (b)).

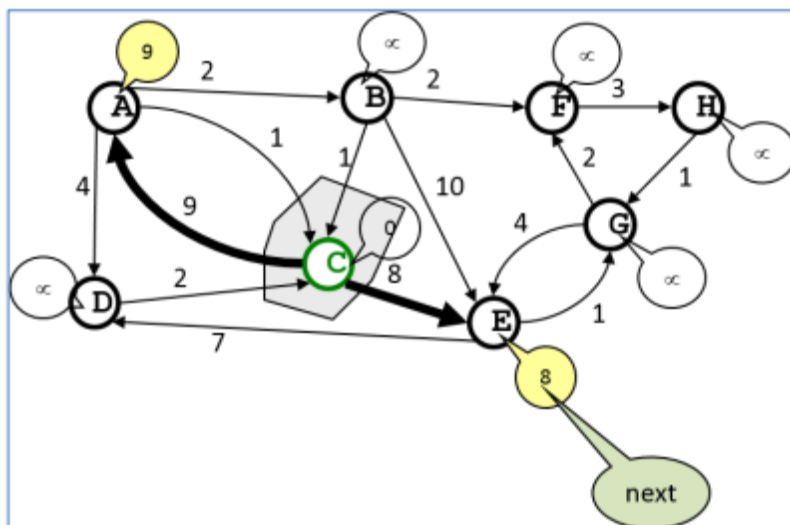


Figure 39.13 (b)

Now vertex E is added to the cloud and we update labels of neighbors of E – here D is updated to 15 (C-E-D that is $8+7=15$) and G to 9 (C-E-G that is $8+1=9$). Since distance to A is the shortest from C that will be the next node to be considered (Figure 39.13 (c)).

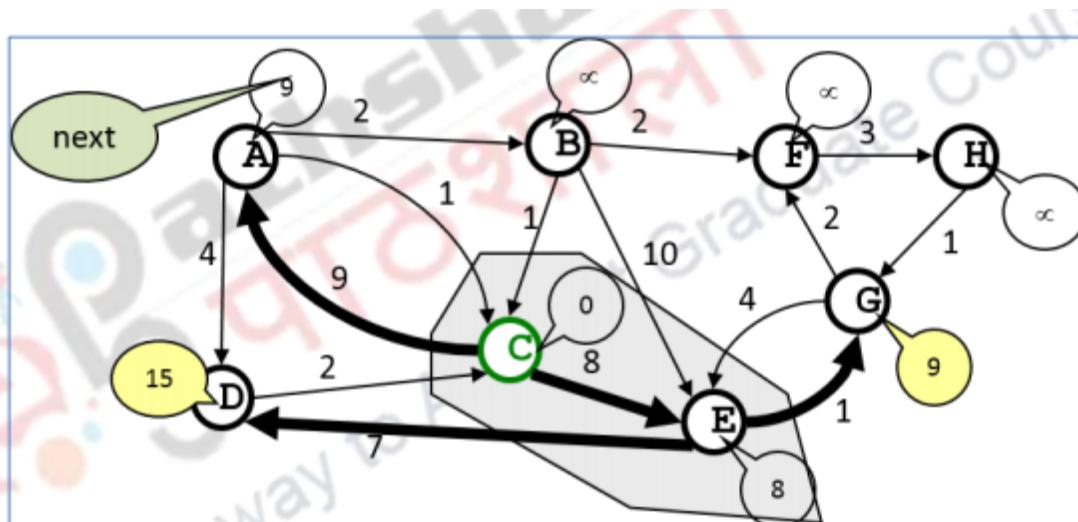


Figure 39.12 (c)

Now vertex A is added to the cloud and we update labels of neighbors of A – here B is updated to 11 (C-A-B that is $9+2=11$) and D's label is revised from 15 to 13 through new path (C-A-D that is $9+4=13$). Since distance to G is the shortest from C that will be the next node to be considered (Figure 39.13 (d)).

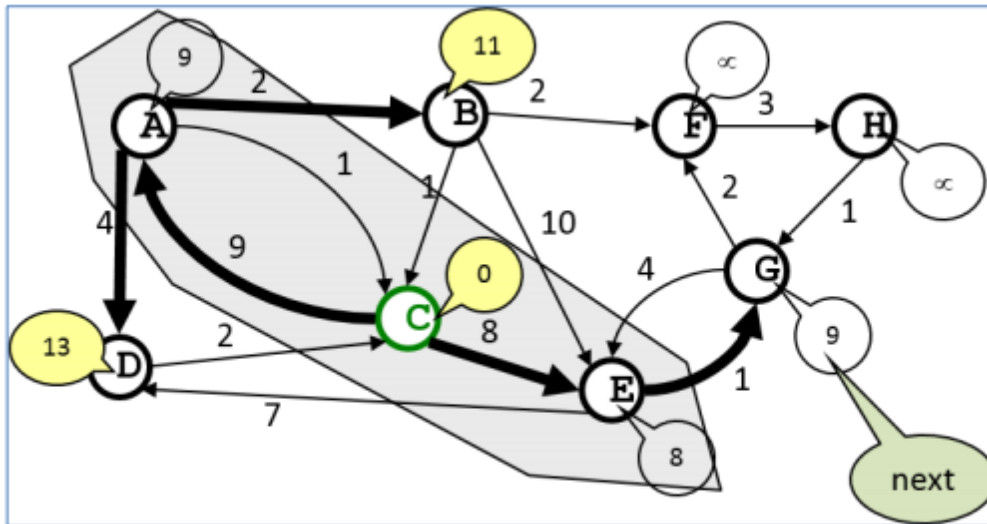


Figure 39.13 (d)

Now vertex G is added to the cloud and we update labels of neighbors of G – here F is updated to 11 (C-E-G-F that is $8+1+2=11$). Since distance to B is one of the shortest from C that will be the next node to be considered (Figure 39.13 (e)).

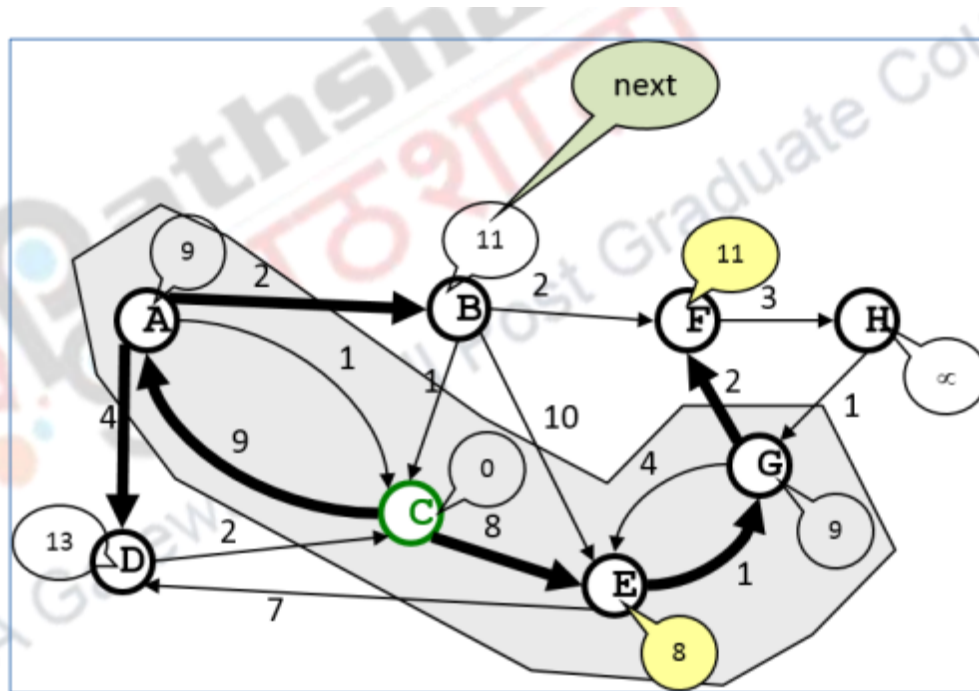


Figure 39.12 (e)

Now vertex B is added to the cloud but no updating takes place. Since distance to F is the shortest from C that will be the next node to be considered (Figure 39.13 (f)).

Now vertex F is added to the cloud and we update labels of neighbors of F – here H is updated to 14 (C-E-G-F-H that is $8+1+2+3=14$). Since distance to D is the shortest from C that will be the next node to be considered (Figure 39.13 (g)).

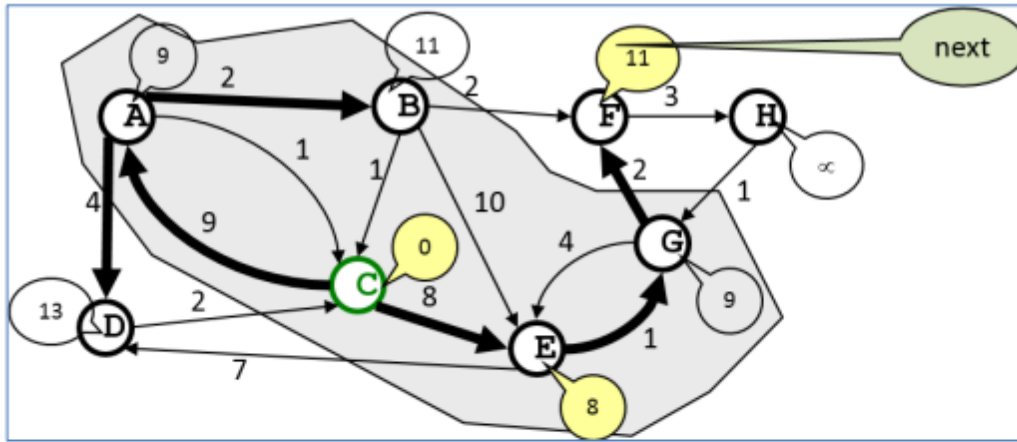


Figure 39.13 (f)

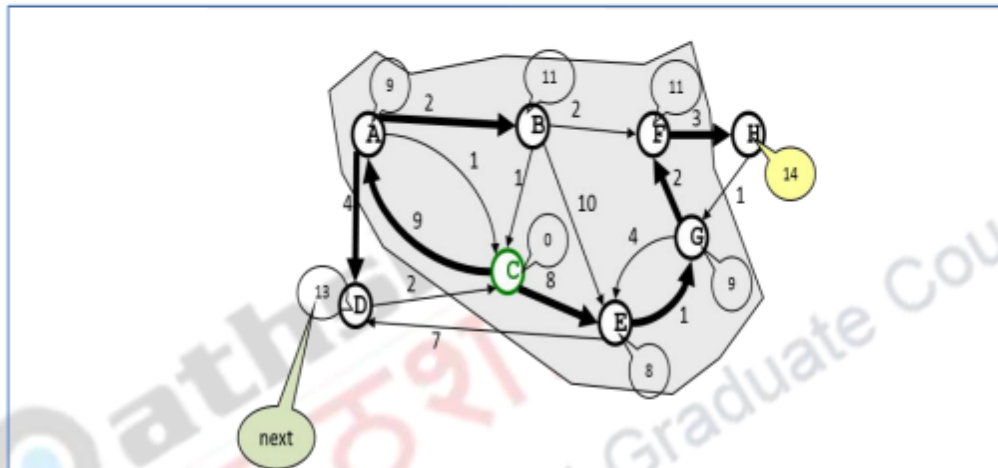


Figure 39.13 (g)

Now vertex D is added to the cloud but no updating takes place. Since distance to H is the shortest from C that will be the next node to be considered (Figure 39.13 (h)).

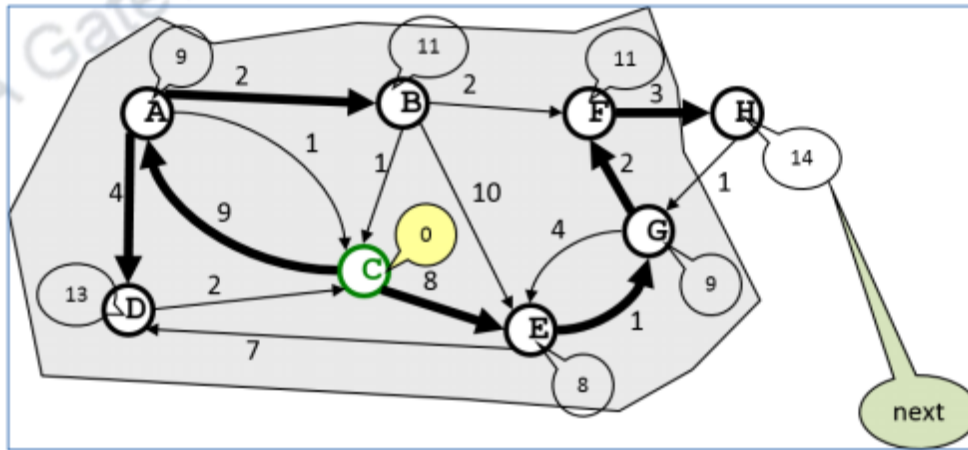


Figure 39.13 (h)

Now vertex H is added to the cloud and since all the vertices of the graph have been visited the algorithm completes (Figure 39.13 (i)).

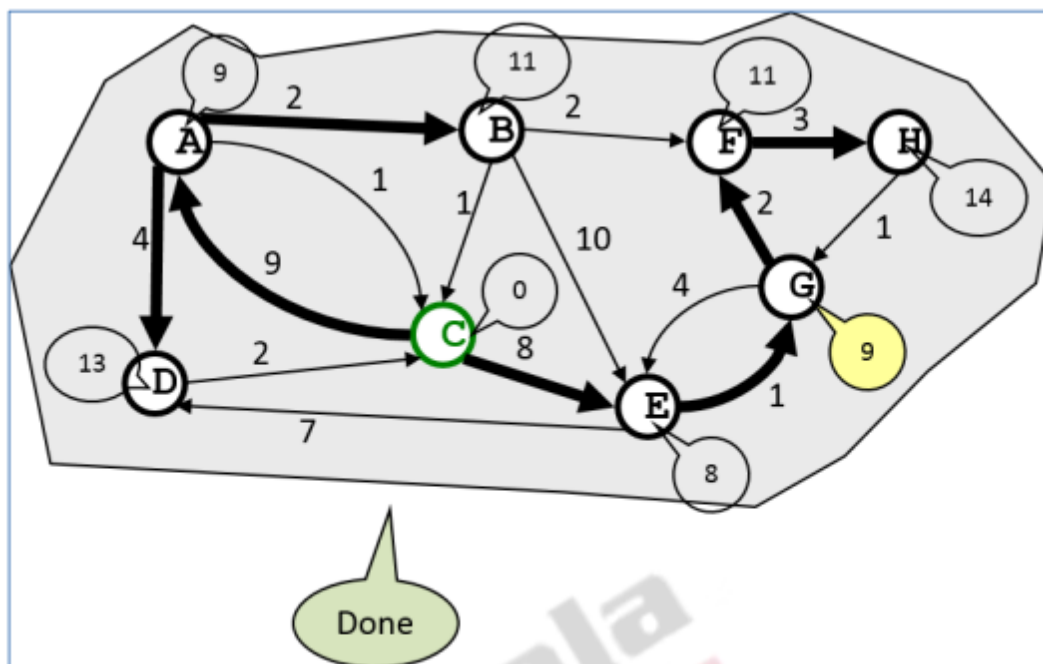


Figure 39.13 (i)

Summary

- Explained the Shortest Path problem and its variants
- Discussed the properties of Shortest path
- Outlined the Dijkstra's Algorithm
- Illustrated the Dijkstra's Algorithm with an example

you can view video on Shortest Path Algorithm – I



Web Links
https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm
https://en.wikipedia.org/wiki/Shortest_path_problem
https://courses.cs.washington.edu/courses/cse373/01sp/Lect24_2up.pdf
http://www.eecs.wsu.edu/~ananth/CptS223/Lectures/shortestpath.pdf
https://www.cs.princeton.edu/~rs/AlgsDS07/15ShortestPaths.pdf
https://www.cise.ufl.edu/~sahni/cop3530/slides/lec326.pdf
math.mit.edu/~rothvoss/18.304.3PM/Presentations/1-Melissa.pdf
https://courses.cs.washington.edu/courses/cse326/03wi/.../lecture21b.ppt
Supporting & Reference Materials
Mark Allen Weiss, "Data Structures and Algorithm Analysis in Java", Pearson; 3rd Edition, 2011
Carrano and Henry, "Data Structures and Problem Solving with C++: Walls and Mirrors", Pearson; 6 edition, 2012
Adam Drozdek, "Data Structures and Algorithms in C++", Cengage; 4th edition, 2013
Alfred V. Aho and Jeffrey D. Ullman, "Data Structures and Algorithms", 1983
Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser, "Data Structures and Algorithms in Java", Wiley; 6 edition, 2014